WEMBRO — CTO INTELLIGENCE BRIEF

Complete Mental Model of the Company

Reverse-engineered from source. Treat as a living document.

EXECUTIVE OVERVIEW

Wembro (codebase name: `rtoshield-control-room`) is a B2B SaaS profit-recovery control room for Indian D2C e-commerce sellers. The first wedge is a CSV-first RTO/NDR loss-prevention workflow sold as a 14-day pilot. The core operating model is:

> DATA → INSIGHT → DECISION → ACTION → LEARNING

The product finds money leaking after checkout (failed COD deliveries → returned shipments), tells operations teams exactly what to fix today, rescues NDR cases before they become permanent RTO losses, and proves rupees saved. It is not a shipping aggregator, COD confirmation app, chatbot, or CRM.

The business engine in one sentence : Sellers upload order/shipment/NDR CSVs → Wembro scores every COD order for RTO risk → builds a daily action queue → rescues NDR cases via mock WhatsApp → records estimated savings → proves ROI → converts to ₹2,999–14,999/month subscription.

PHASE 1 — FULL SYSTEM DISCOVERY

1. Product Surface Area

All Routes & Pages

Route	Type	Purpose	Plan Gate
`/`	Public	Marketing homepage + live ROI calculator	Free
`/login`	Public	Supabase auth (signup/signin)	Free
`/product`	Public	Product feature overview	Free
`/pricing`	Public	Pricing tiers	Free
`/calculator`	Public	Free RTO Loss Calculator	Free
`/sample-report`	Public	Fictional Nazrana Streetwear demo audit	Free
`/audit`	Public	Privacy-safe audit flow (summary or anon CSV)	Audit ₹999
`/pilot`	Public	14-day pilot workflow tracker	Pilot ₹4,999
`/demo`	Auth	Client test mode with generated data	All plans
`/dashboard`	Auth	**Main control room (57KB+ component)**	Starter+
`/personas/founder`	Public	Founder persona marketing page	Free
`/personas/growth-lead`	Public	Growth lead persona page	Free
`/personas/operations`	Public	Ops persona page	Free

Dashboard Internal Views (single-page, state-driven)

- Profit Cockpit, CSV Upload, Order Risk, NDR Rescue, Daily Action Queue
- Messaging Outbox, Savings Ledger, Leakage Report, Pincode Intelligence
- Courier Intelligence, SKU Intelligence, Campaign Intelligence
- Policy Simulator, Weekly Founder Report, Monthly Strategy Report
- Brand Settings, Stores, Custom Rules, NDR Playbooks
- Integration Readiness, Onboarding, SOPs, Privacy & Audit, Plan & Billing

---

2. Frontend Architecture

Framework: Next.js 15 (App Router) + React 19

Component strategy: "use client" for dashboard and homepage. Server components only for layout.

State management: No Redux/Zustand. All state lives in:

1. React `useState` / `useReducer` inside the 57KB dashboard component
2. `localStorage` via `workspaceStorage.ts` (single workspace key)
3. `useAppData` hook (fetches from API via `Promise.allSettled`)

Key architectural concern: The dashboard (`src/app/dashboard/page.tsx`, 57KB+) is a monolithic client component with 50+ imports and 8+ main views. It is the single biggest complexity bottleneck in the codebase.

UI component system (`src/components/ui/controlRoom.tsx`):

- MetricCard, InsightCard, ActionCard
- RiskBadge, PriorityBadge, StatusBadge, ConfidenceBadge
- EmptyState, PageHeader, DataQualityBadge
- MoneyValue, PercentValue, PhoneMasked
- RecommendationPanel, Timeline, FilterBar
- DrawerDetailLayout, LoadingSkeleton, UpgradeGate, DemoModeBanner

Rendering model:

- Public pages: SSR/SSG where possible
- Dashboard: fully client-rendered, data fetched client-side via `useAppData`
- No streaming, no Suspense boundaries, no server actions used

---

3. Backend Architecture

Runtime: Next.js API routes (`src/app/api/v1/`)

All API routes: Force-dynamic, JSON-only, Supabase-auth-gated

Repository pattern: 7 repositories (Order, NDR, Action, Savings, Brand, User, AuditLog)

Validation: Zod schemas in `src/backend/schemas/api.schema.ts` (10 schemas)

Business logic layer: `src/lib/` (pure functions, no side effects except events)

Orchestration: 13 connectors in `src/shared/connectors/` wire features together

No background jobs. No queues. No cron. No webhooks. All synchronous.

---

4. Database & Data Models

Prisma Schema (6 tables — PRODUCTION DEPLOYED)

Table	Key Fields	Relationships
`brands`	id, name, createdAt	→ users, orders, ndrCases, actions, savingsEvents, auditLogs
`users`	id (= Supabase auth.users.id), brandId, email, role	→ brand, auditLogs
`orders`	brandId, orderId, awb, status, riskScore, riskLevel, codAmount, paymentMode	→ brand, ndrCase?, actions[]
`ndr_cases`	brandId, orderId (unique), reason, status	→ brand, order, actions[]
`actions`	brandId, orderId?, ndrCaseId?, type, status	→ brand, order?, ndrCase?
`savings_events`	brandId, type, amount, status (default: estimated)	→ brand
`audit_logs`	brandId, userId?, action, targetId?, details (JSON)	→ brand, user?

DATA_MODEL.md (14 tables — INTENDED SCHEMA, NOT YET DEPLOYED)

The intended schema includes 8 additional tables not in Prisma: `imports`, `customers`, `risk_scores`, `address_checks`, `messages`, `customer_responses`, `brand_members`, `stores`

This is the biggest architectural gap in the codebase. The Prisma schema is ~40% of the intended data model. The missing tables mean: no server-side import tracking, no customer history, no persistent message records, no address check persistence, no multi-store support at the DB level.

localStorage Schema (CURRENT MVP STATE LAYER)

```
Key: rtoshield:starter_v1 (or pro_v1)
Contains:
  brand (BrandSettings)
  orders (Order[])
  imports (ImportRecord[])
  ndrCases (NdrCase[])
  messages (Message[])
  responses (CustomerResponse[])
  actions (ActionItem[])
  savingsEvents (SavingsEvent[])
  audits (AuditLog[])
  stores? (Store[])
  policyRecommendations?
  weeklyReports?
  monthlyStrategyReports?
  policySimulations?
  exports?
```

The app runs almost entirely off localStorage. The Prisma/Supabase DB is used only for auth context, brand record, orders list, NDR list, actions list, savings, and audit logs. There are two competing state stores with no sync.

---

5. Intelligence / AI Layer

All intelligence is **deterministic rule-based**, not ML. Explicitly designed to be explainable.

Engine	Location	Algorithm	Output
Risk Scoring	`lib/riskScoring.ts`	Weighted additive, 0–100, capped	score, bucket, reasons[], recommendedAction
Advanced Risk	`features/risk/advancedRisk.service.ts`	Base score + margin/discount/campaign modifiers + custom rules	Same + customRuleMatches[], confidenceLabel
Address Quality	`lib/addressQuality.ts`	Penalty deduction from 100	score, issues[], suggestedQuestion
NDR Normalization	`lib/ndr.ts`	Pattern matching → 12 categories	reason, confidence (0.45–0.9), recommendedAction
NDR Urgency	`lib/ndrCases.ts`	SLA elapsed time + attempt count	urgency: critical/high/medium/low
Intent Detection	`lib/responses.ts`	Keyword/phrase matching	intent, confidence
Prepaid Scoring	`features/prepaid/service.ts`	Multi-factor additive score	opportunity, maxSafeIncentive
Leakage Atlas	`features/leakage-atlas/service.ts`	7 driver analysis	top driver, route, leakage estimate
Policy Simulator	`features/policy-simulator/service.ts`	Scenario-based net benefit calc	netEstimatedBenefit
Profit Recovery	`lib/profitRecovery.ts`	Formula: fwd+rtn+pkg+CAC+COD fee	estimatedLeakage per order

Risk Score Components:

Signal	Points	Notes
COD payment mode	+25	Base COD risk
Missing/weak address	+20	< 35 chars

Signal	Points	Notes
Pincode RTO rate > 30%	+20	From current dataset
Previous customer RTO	+25	Repeat bad actor
High-value COD ≥ ₹3,999	+20	Value threshold
Attempt count ≥ 3	+20	Courier persistence
Courier RTO > 25%	+15	Lane risk
Short address < 35 chars	+15	Specificity
NDR reason penalty	+8–25	Override by reason type

---

6. Integrations

Integration	Status	Provider Options
Supabase Auth	✅ Live	Supabase
Supabase Postgres	✅ Live (partial)	Supabase
WhatsApp	⚠️ Mock only	meta_cloud, gupshup, wati, interakt, aisensy (future)
Shopify	⚠️ Placeholder	—
WooCommerce	⚠️ Placeholder	—
Shiprocket	⚠️ Placeholder	—
NimbusPost	⚠️ Placeholder	—
Delhivery	⚠️ Placeholder	—
Payment links	⚠️ Placeholder	Razorpay/Cashfree (future)
Helpdesk	⚠️ Placeholder	—
Accounting	⚠️ Placeholder	—
Email	❌ Not built	—

---

7. Infrastructure

Concern	Implementation
Hosting	Vercel (inferred from next.config.ts `cpus:1, workerThreads:false`)
Database	Supabase Postgres via `@prisma/adapter-pg`
Auth	Supabase Auth + SSR (`@supabase/ssr`)
Storage	Browser localStorage (MVP), Postgres (partial)
CDN	Vercel Edge
Queues	None
Cron	None

Observability	None
Concern	**Implementation**
Rate limiting	None
Caching	None (all routes `force-dynamic`)
Build	`prisma generate && next build`
TypeScript errors	Suppressed (`ignoreBuildErrors: true`) ⚠
ESLint	Disabled during builds ⚠

---

8. Security

Area	Status	Notes
Authentication	✅ Supabase JWT	Standard, managed
Multi-tenant isolation	✅ brandId scoping	All repos scope to brandId
Phone masking	✅	maskPhone(), phoneHash()
RBAC (roles exist)	⚠ Partial	4 roles defined, NOT enforced in repositories
RBAC (data layer)	❌ Missing	canRole() only used in UI, not API
PII in localStorage	❌ Unencrypted	Names, phones, addresses stored in browser
Rate limiting	❌ Missing	CSV import has no size/rate limit
TypeScript suppression	❌ Risk	`ignoreBuildErrors: true` hides type errors
Soft deletes	❌ Missing	Hard deletes cascade, no recovery
Audit log tamper protection	❌ Missing	No HMAC, deletable
Secrets in code	✅ Clean	All via env vars

PHASE 2 — MASTER SYSTEM MAP

A. High-Level SaaS Architecture

```
graph TB
    subgraph "Public Surface"
        HP[Homepage /]
        CALC[Calculator /calculator]
        AUDIT[Audit /audit]
        PILOT[Pilot /pilot]
        SAMPLE[Sample Report]
        LOGIN[Login /login]
    end

    subgraph "Authenticated App"
        DASH[Dashboard /dashboard\n57KB monolith]
        DEMO[Demo Mode]
    end

    subgraph "API Layer /api/v1/"
        OR[Orders API]
```

```
    NA[NDR API]
    AC[Actions API]
    SA[Savings API]
    BR[Brand API]
    AU[Audit API]
    US[Users/Me API]
end

subgraph "Intelligence Layer"
    RISK[Risk Engine\n0-100 score]
    NDR[NDR Engine\n12 reasons]
    ADDR[Address Engine\n0-100 score]
    PREP[Prepaid Engine\nopportunity score]
    LEAK[Leakage Atlas\n7 drivers]
    POL[Policy Simulator\n6 scenarios]
end

subgraph "Data Layer"
    LS[localStorage\nMVP state]
    PG[(Supabase Postgres\n6 tables)]
    SA2[Supabase Auth]
end

subgraph "External Systems"
    WA[WhatsApp\nMOCK ONLY]
    SHOP[Shopify\nPLACEHOLDER]
    COUR[Courier APIs\nPLACEHOLDER]
end

LOGIN --> SA2
SA2 --> DASH
HP --> LOGIN
CALC --> LS

DASH --> OR
DASH --> NA
DASH --> AC
DASH --> SA
DASH --> BR
DASH --> AU

OR --> PG
NA --> PG
AC --> PG
SA --> PG
BR --> PG
AU --> PG

DASH --> RISK
DASH --> NDR
DASH --> ADDR
DASH --> PREP
DASH --> LEAK
DASH --> POL

RISK --> LS
NDR --> LS
PREP --> LS
LEAK --> LS

DASH --> WA
DASH -.->|future| SHOP
DASH -.->|future| COUR
```

B. Data Flow Diagram

```
flowchart TD
    CSV[CSV Upload\nOrder / Shipment / NDR data]
    PARSE[Parse + Normalize\ncsvImport.ts\n45+ field aliases]
    QUALITY[Data Quality Score\n0-100, per area]
    RISK_SCORE[Risk Scoring\nscoreOrder × N orders]
    ADDR_CHECK[Address Quality\ncheckAddressQuality × N]
    NDR_DETECT[NDR Detection\nbuildNdrCases]
    NORM[NDR Reason Normalization\nnormalizeNdrReason × M cases]
    LEAKAGE[Leakage Estimation\nestimatedLeakageForOrder]
    DRIVERS[Leakage Atlas\n7 drivers ranked]
    ACTION_Q[Daily Action Queue\nbuildAdvancedActionQueue\n9 groups]
    POLICY[Policy Recommendations\n4 policy types]
    PREPAID[Prepaid Opportunities\nfindAdvancedPrepaidOpportunities]
    MSG[Message Queuing\nqueueMockMessage]
    RESP[Customer Response\ndetectIntent]
    OUTCOME[Outcome Recording\nmark delivered/RTO/cancel]
    SAVINGS[Savings Events\n6 event types]
    ROI[ROI Aggregation\ncalculateRoi]
    REPORT[Reports\nWeekly + Monthly + Audit]
    LEARN[Learning Loop\nlabeled data for future ML]

    CSV --> PARSE --> QUALITY
    PARSE --> RISK_SCORE --> LEAKAGE
    PARSE --> ADDR_CHECK
    PARSE --> NDR_DETECT --> NORM
    LEAKAGE --> DRIVERS
    DRIVERS --> ACTION_Q
    ACTION_Q --> POLICY
    ACTION_Q --> PREPAID
    ACTION_Q --> MSG
    MSG --> RESP --> OUTCOME
    OUTCOME --> SAVINGS --> ROI --> REPORT
    REPORT --> LEARN
    NORM --> ACTION_Q
    ADDR_CHECK --> ACTION_Q
```

C. Feature Dependency Graph

```
graph LR
    subgraph "Core Domain"
        IMP[imports]
        ORD[orders]
        RISK[risk]
        NDR[ndr]
        ADDR[address]
    end

    subgraph "Decision Engine"
        ACT[actions]
        RUL[rules]
        PREP[prepaid]
        POL_R[policy-recommendations]
        POL_S[policy-simulator]
        MIS[missions]
        LEAK[leakage-atlas]
    end

    subgraph "Intelligence"
        PIN[pincode]
        COU[courier]
        SKU[sku]
```

```
    CAM[campaigns]
end

subgraph "Reporting"
    ROI[roi]
    SAV[savings-ledger]
    RPT[reports]
    WKR[weekly-report]
    MNT[monthly-strategy]
end

subgraph "Infrastructure"
    MSG[messaging]
    RSP[responses]
    BRD[brand]
    PLN[plans]
    ROL[roles]
    STO[stores]
end

subgraph "Lib Layer"
    LR[lib/riskScoring]
    LN[lib/ndr + ndrCases]
    LC[lib/csvImport]
    LP[lib/profitRecovery]
    LA[lib/addressQuality]
    LI[lib/roi]
    LM[lib/messaging]
    LRS[lib/responses]
    LAG[lib/actionGroups]
end

IMP --> LC
IMP --> PLN
ORD --> |events| shared/events
RISK --> LR
RISK --> RUL
RISK --> LI
NDR --> LN
ADDR --> LA
LEAK --> LP
LEAK --> LAG
ACT --> LP
ACT --> LAG
MIS --> LP
MIS --> LAG
POL_R --> LP
POL_S --> LP
PREP --> LI
PIN --> LP
COU --> LP
SKU --> LP
CAM --> LP
ROI --> LI
SAV --> LM
WKR --> LI
MNT --> LI
MSG --> LM
MSG --> LAG
RSP --> LRS
```

D. Operational Intelligence Map

```
graph TD
    subgraph "Leakage Detection"
        COD_RISK[COD Risk Detection\nHigh/Critical COD orders\nscoreOrder > 60]
        NDR_SLA[NDR SLA Monitor\n>= 8h elapsed\nattemptCount signals]
        ADDR_WEAK[Weak Address\naddressScore < 60\nmissing landmark/city]
        FAKE_ATTEMPT[Courier Fake Attempt\nndr reason = courier_fake_attempt]
    end

    subgraph "Profit Intelligence"
        RTO_LOSS[RTO Loss Formula\nfwd + rtn + pkg + CAC + COD fee\nper brand settings]
        RECOVER[Recoverable Leakage\nHigh/Critical × leakage multiplier]
        SAVINGS_PROOF[Savings Events\n6 types, estimated/verified]
        NET_BENEFIT[Net Benefit\nsavings - software cost - msg cost]
    end

    subgraph "Courier Intelligence"
        COU_RTO[Courier RTO Rate\ngrouped by courier\nconfidence thresholds]
        COU_REC[Courier Recommendation\nmonitor / switchback / review]
        LANE[Failing Lane\ncourier × pincode cluster]
    end

    subgraph "SKU Intelligence"
        SKU_RTO[SKU RTO Rate\ngrouped by sku\n>= 10 order threshold]
        SKU_REASON[Dominant Reason\nrefusal vs wrong address\nvs fake attempt]
        SKU_REC[SKU Recommendation\nproduct/targeting/pricing]
    end

    subgraph "Pincode Intelligence"
        PIN_RTO[Pincode RTO Rate\n> 30% → verify COD]
        PIN_COD_RTO[COD RTO Rate\n> 35% → prepaid only]
        PIN_ADDR[Wrong Address Rate\n> 35% → correction workflow]
        PIN_REC[Pincode Recommendation\nmonitor/verify/prepaid-only]
    end

    subgraph "Campaign Attribution"
        UTM[UTM / Campaign grouping\ncampaignName OR utmCampaign\nOR sourcePlatform OR adId]
        CAM_RTO[Campaign RTO Rate\n> 30% + COD > 60%]
        CAM_REC[Campaign Recommendation\ntest prepaid nudge / review ad-product match]
    end

    subgraph "Trust Systems"
        DATA_TRUST[Data Trust\nready/limited/blocked\nper analysis area]
        CONF[Confidence Labels\nhigh/medium/low]
        LOW_SAMPLE[Low-Sample Warnings\n< 10 orders threshold]
    end

    COD_RISK --> RECOVER
    NDR_SLA --> RECOVER
    ADDR_WEAK --> RECOVER
    RECOVER --> RTO_LOSS
    RTO_LOSS --> NET_BENEFIT
    SAVINGS_PROOF --> NET_BENEFIT
    DATA_TRUST --> LOW_SAMPLE
    CONF --> LOW_SAMPLE
```

E. Automation Graph (Connector Layer)

```
flowchart LR
    subgraph "Triggers"
        T1[CSV Import]
        T2[Order Created]
        T3[Order Updated]
        T4[NDR Detected]
        T5[Customer Response]
        T6[Outcome Confirmed]
        T7[Analytics Run]
        T8[Report Request]
        T9[Export Request]
    end

    subgraph "Connectors (Orchestration)"
        C1[importPipeline]
        C2[orderToRisk]
        C3[orderToNdr]
        C4[orderToAddress]
        C5[orderToActions]
        C6[orderToCustomRules]
        C7[orderToPrepaid]
        C8[actionToMessaging]
        C9[responseToAction]
        C10[policyRecommendation]
        C11[reporting]
        C12[savings]
        C13[export]
    end

    subgraph "Outputs"
        O1[Risk Scores]
        O2[NDR Cases]
        O3[Address Checks]
        O4[Action Queue]
        O5[Rule Matches]
        O6[Prepaid Opportunities]
        O7[Queued Messages]
        O8[Next Actions]
        O9[Policy Recs]
        O10[Reports]
        O11[Savings Events]
        O12[Report Package]
    end

    T1 --> C1 --> O1
    C1 --> O2
    T2 --> C2 --> O1
    T2 --> C3 --> O2
    T2 --> C4 --> O3
    T2 --> C5 --> O4
    T2 --> C6 --> O5
    T2 --> C7 --> O6
    T4 --> C8 --> O7
    T5 --> C9 --> O8
    T7 --> C10 --> O9
    T8 --> C11 --> O10
    T6 --> C12 --> O11
    T9 --> C13 --> O12
```

F. Integration Graph

```
graph LR
    subgraph "Live "
        SUP_AUTH[Supabase Auth\nJWT + metadata]
        SUP_PG[Supabase Postgres\nPrisma adapter]
    end

    subgraph "Mock / Planned "
        WA_MOCK[WhatsApp\nMock outbox\nmanual export]
        WA_REAL[WhatsApp Real\nmeta_cloud / gupshup\nwati / interakt / aisensy]
    end

    subgraph "Placeholder "
        SHOP[Shopify\nwebhook receiver]
        WC[WooCommerce\nwebhook receiver]
        SR[Shiprocket\nNDR + status API]
        NP[NimbusPost\nNDR + status API]
        DEL[Delhivery\nNDR + status API]
        PAY[Payment Links\nRazorpay / Cashfree]
        HELP[Helpdesk\nZendesk / Freshdesk]
        ACC[Accounting\nTally / Zoho]
    end

    APP[Wembro App] --> SUP_AUTH
    APP --> SUP_PG
    APP --> WA_MOCK
    WA_MOCK -.->|Phase 2| WA_REAL
    APP -.->|Phase 3| SHOP
    APP -.->|Phase 3| WC
    APP -.->|Phase 4| SR
    APP -.->|Phase 4| NP
    APP -.->|Phase 4| DEL
    APP -.->|Phase 2+| PAY
    APP -.->|Future| HELP
    APP -.->|Future| ACC
```

G. Database ERD (Prisma — Deployed)

```
erDiagram
    brands {
        uuid id PK
        string name
        datetime created_at
        datetime updated_at
    }

    users {
        uuid id PK
        uuid brand_id FK
        string email
        string role
        datetime created_at
        datetime updated_at
    }

    orders {
        uuid id PK
        uuid brand_id FK
        string order_id
        string awb
        string customer_phone
        string status
        int risk_score
        string risk_level
```

```
        string risk_level
        float cod_amount
        string payment_mode
        datetime created_at
        datetime updated_at
    }

    ndr_cases {
        uuid id PK
        uuid brand_id FK
        uuid order_id FK
        string reason
        string status
        datetime created_at
        datetime updated_at
    }

    actions {
        uuid id PK
        uuid brand_id FK
        uuid order_id FK
        uuid ndr_case_id FK
        string type
        string status
        datetime created_at
        datetime updated_at
    }

    savings_events {
        uuid id PK
        uuid brand_id FK
        string type
        float amount
        string status
        datetime created_at
    }

    audit_logs {
        uuid id PK
        uuid brand_id FK
        uuid user_id FK
        string action
        uuid target_id
        json details
        datetime created_at
    }

    brands ||--o{ users : "has"
    brands ||--o{ orders : "owns"
    brands ||--o{ ndr_cases : "owns"
    brands ||--o{ actions : "owns"
    brands ||--o{ savings_events : "owns"
    brands ||--o{ audit_logs : "owns"
    orders ||--o| ndr_cases : "has"
    orders ||--o{ actions : "has"
    ndr_cases ||--o{ actions : "has"
    users ||--o{ audit_logs : "creates"
```

H. Deployment & Runtime Architecture

```
graph TB
    subgraph "Client (Browser)"
        REACT[React 19 App\nNext.js 15 Client]
        LS[localStorage\nFull workspace state\norders, NDR, messages, savings]
        RISK_CLIENT[Risk Engine\nClient-side compute]
    end

    subgraph "Vercel Edge"
        MW[middleware.ts\nSupabase session refresh\nall routes except static/images]
    end

    subgraph "Vercel Serverless Functions"
        API_V1[/api/v1/* routes\nOrders, NDR, Actions\nSavings, Brand, Audit, Users]
        PRISMA[Prisma Client\nPgAdapter]
    end

    subgraph "Supabase"
        AUTH[Auth Service\nJWT + user_metadata\nbrandId stored here]
        PG[(PostgreSQL\n6 tables\npartial schema)]
    end

    REACT --> MW
    MW --> REACT
    REACT --> API_V1
    API_V1 --> PRISMA --> PG
    API_V1 --> AUTH
    REACT --> AUTH
    REACT --> LS
    REACT --> RISK_CLIENT
```

PHASE 3 — SYSTEM ANALYSIS

1. Architectural Weaknesses

#	Issue	Severity	Impact
1	**Dual state stores** — localStorage + Prisma DB with no sync strategy	Critical	Data inconsistency, audit trail gaps
2	**Prisma schema is 40% of intended model** — 6 of 14 planned tables missing	Critical	No server-side import/message/customer/address records
3	**Monolithic 57KB dashboard component** — all views in one file, 50+ imports	Critical	Unmaintainable, slow to test, renders everything on mount
4	**TypeScript errors suppressed** — `ignoreBuildErrors: true`	Critical	Silent type bugs in production
5	**RBAC exists in UI only** — roles defined but not enforced in API/repositories	High	Ops users can call admin endpoints directly
6	**No persistent event store** — events in localStorage, lost on logout	High	Audit trail incomplete, no replay
7	**Hard deletes with cascade** — no soft deletes anywhere	High	No recovery, violates audit requirements
8	**No input validation in scoring functions** — negative values, null context	Medium	Silent wrong scores
9	**ESLint disabled in builds**	Medium	Code quality drift
10	**next.config.ts: cpus:1, workerThreads:false** — performance capped	Medium	Slow builds, no parallelism

2. Scalability Bottlenecks

Bottleneck	Location	Why It Matters
O(N²) risk scoring	`csvImport.ts`	Pincode RTO rate recalculated per order across all orders. 10k rows = 100M comparisons
No pagination	All repositories' `getByBrandId()`	Full table scan for every request, memory exhaustion at scale
localStorage ceiling	~5–10MB browser limit	Pro plan at 10k orders hits this; JSON serialization overhead
batchUpsert = N separate DB calls	`OrderRepository.batchUpsert()`	1,000 orders = 1,000 round trips, no bulk INSERT
`force-dynamic` on all routes	`next.config.ts`	No caching anywhere, CDN unused
useAppData: no pagination, no streaming	`lib/api/use-app-data.ts`	Loads all data before any render
Single monolith build worker	`next.config.ts`	`webpackBuildWorker: false`, `cpus: 1`

3. Dangerous Coupling

Coupling	Risk
Dashboard imports 50+ modules directly	Change in any feature can break the dashboard silently
`getAuthContext()` stores `brandId` in Supabase `user_metadata`	If metadata is missing, user is locked out with a 400 error
CSV import pipes directly into in-memory risk scoring	Large CSV blocks the browser main thread
Connectors call features directly without validation	Null inputs cause exceptions not caught at connector boundary
`demo.demoDataGenerator.ts` imports from 4 lib modules	Demo data generation is tightly coupled to scoring logic; any formula change affects demo

4. Redundant Logic / Feature Duplication

Duplication	Location
`isNdrOrder()` defined in `lib/ndrCases.ts` AND re-exported from 3 features	`features/ndr`, `features/actions`, `features/leakage-atlas`
`estimatedLeakageForOrder()` from `lib/profitRecovery` imported by 8+ features independently	Should be centralized via a single analytics service
`calculateRoi()` from `lib/roi` re-exported through `features/roi/roi.service.ts` with no added logic	Thin wrapper with zero value
Address quality checks run in both `lib/addressQuality.ts` (standalone) and inside `riskScoring.ts`	Potential divergence in scoring
NDR reason normalization runs at CSV import AND again in `features/ndr/ndr.service.ts`	Double processing

5. Technical Debt Hotspots

File	Debt

File	Debt
`src/app/dashboard/page.tsx` (57KB)	Monolithic component; needs decomposition into route segments
`src/lib/csvImport.ts` (427 lines)	No transaction, O(N²) scoring, no streaming
`src/lib/riskScoring.ts` (254 lines)	Hardcoded thresholds (₹999, ₹1499, ₹2499, ₹3999); should be brand config
`src/lib/ndrCases.ts`	12h SLA hardcoded; NDR state transitions not validated
`prisma/schema.prisma`	6 tables; 8 tables missing vs DATA_MODEL.md; no migration for missing tables
`next.config.ts`	`ignoreBuildErrors: true` is a silent debt bomb

---

6. Slow Query Risks

Query	Risk
`getByBrandId()` on orders — no pagination	Full scan per request
batchUpsert loop — 1 query per order	N round-trips for N orders
AuditLog hardcoded `limit: 100`	Not configurable, may miss events
No indexes beyond Prisma's unique constraints	Pincode/courier/sku group-by operations unindexed

Recommended indexes (from DATA_MODEL.md, not applied): `(brand_id, order_id)`, `(brand_id, awb)`, `(brand_id, pincode)`, `(brand_id, payment_mode)`, `(brand_id, risk_bucket)`, `(brand_id, final_status)`

---

7. Dead Code / Unused Infrastructure

Item	Notes
`src/features/learning/`	Only exports help content; no ML infrastructure
`src/features/marketing/`	Only exports marketing copy strings
`integration-readiness` cards	All 6 are static placeholders, no live checks
WhatsApp provider abstractions	`meta_cloud`, `gupshup`, `wati`, `interakt`, `aisensy` defined but never called
`personas/*` pages	Marketing pages with no functional interaction
`lib/leadStore.ts`	localStorage-only lead capture; not connected to any backend

---

PHASE 4 — BUSINESS INTELLIGENCE VIEW

What The Company Actually Does

Wembro is a **profit recovery operating system** for Indian D2C brands. The core value is: take 30 days of operational data, find where money is leaking, tell the ops team what to do today, rescue shipments before they become losses, and prove the savings.

The first commercial wedge is the 14-day RTO/NDR pilot: sellers pay ₹4,999 to see whether the workflow works before committing to a monthly subscription.

The Core Business Engine

```
Free Calculator → Captures email + pain level
    ↓
Audit (₹999) → First data touch, anonymized CSV
    ↓
14-Day Pilot (₹4,999) → Proves savings with real orders
    ↓
Starter ₹2,999/month → Ops workflow habit formed
    ↓
Growth ₹7,999/month → Full intelligence, team usage
    ↓
Pro ₹14,999/month → Founder cockpit, strategy reports
    ↓
[Future] Managed/Scale/Enterprise → High-volume, integrations
```

The **conversion flywheel** is: CSV upload → immediate leakage diagnosis → action queue → first saved order → savings event recorded → ROI proven → upgrade.

The Moat

Moat Layer	Current Status	How It Builds
Cross-brand pincode intelligence	Not yet (single-tenant)	Phase 6: aggregated failure rates across all sellers
Courier performance index	Single-seller only	Phase 6: industry-level courier benchmarks
Buyer trust signals	Not yet	Customer history across brands (privacy-constrained)
Category benchmarks	Not yet	"Your RTO is 24%; fashion average is 18%"
Labeled outcome data	Being collected	Required for Phase 5 ML models
SOP library	6 templates present	Proprietary ops playbooks for D2C verticals

The current moat is **workflow stickiness** — once an ops team builds their daily queue habit inside Wembro, switching is operationally costly. The future moat is **network intelligence** — cross-brand data that no individual seller can replicate.

The Automation Strategy

Level	Status	Description
L0: Manual	Done	Seller uploads CSV manually
L1: Insight	Done	Risk scores, leakage drivers, recommended actions
L2: Recommendation	Done	"Call this customer", "Queue WhatsApp", "Hold this order"
L3: Draft Action	Partial	Mock WhatsApp messages pre-populated
L4: Approval-based Execution	Planned Phase 2	Real WhatsApp sent after ops approval
L5: Rule-based Automation	Planned Phase 4	Auto-reattempt, auto-cancel triggers
L6: AI Agents	Phase 7+	Autonomous ops with human override

What Would Break The Company If Removed?

System	Why It's Mission Critical
CSV Import + Risk Scoring	The entire product starts here. No CSV → no data → no insights → no value

pipeline System	Why It's Mission Critical
Daily Action Queue	The habit-forming surface. If it stops working, ops teams disengage immediately
Savings Ledger + ROI proof	This is the commercial proof. Without it, sellers can't justify the subscription
NDR Rescue workflow	The most time-sensitive, highest-dollar-impact feature. NDR has a 12-24h window before it becomes a permanent loss
Supabase Auth + brandId isolation	Remove this and the product is both broken and insecure

---

PHASE 5 — CTO CONTROL CENTER OUTPUT

1. EXECUTIVE OVERVIEW

Wembro is a 34-feature, rule-based, CSV-first profit recovery SaaS built on Next.js 15 + Supabase. The MVP is functionally complete for Starter and Pro plans with realistic demo data, explainable risk scoring, NDR rescue workflows, mock WhatsApp outbox, and a savings ledger. The product is **demo-ready and pilot-ready** but **not production-ready** — the current architecture uses localStorage as the primary state store, the Prisma schema covers only 40% of the intended data model, and there is no RBAC enforcement at the API layer.

---

2. MASTER ARCHITECTURE MAP

See diagrams in Phase 2 above.

---

3. FEATURE INVENTORY

Feature	Category	Complexity	Plan Gate	State Owner
risk	Core Intelligence	High	Starter+	localStorage
ndr	Core Intelligence	High	Starter+	localStorage
imports	Core Intelligence	High	All	localStorage
orders	Core Intelligence	Medium	All	localStorage + DB
leakage-atlas	Core Intelligence	Medium	Starter+	localStorage
actions	Decision Engine	High	Starter+	localStorage
rules	Decision Engine	Medium	Pro	localStorage
prepaid	Decision Engine	Medium	Growth+	localStorage
policy-recommendations	Decision Engine	Medium	Pro	localStorage
policy-simulator	Decision Engine	Medium	Pro	localStorage
missions	Decision Engine	Medium	Starter+	localStorage
roi	Reporting	Low	All	localStorage
savings-ledger	Reporting	Medium	Starter+	localStorage + DB
reports	Reporting	Low	All	localStorage
weekly-report	Reporting	Medium	Growth+	localStorage

Feature	Category	Complexity	Plan Gate	State Owner
reorder-strategy	Reporting	Medium	Pro	localStorage
messaging	Engagement	Medium	Starter+	localStorage + DB
responses	Engagement	Low	Starter+	localStorage
ndr-playbooks	Engagement	Low	Pro	localStorage
pincode	Intelligence	Medium	Growth+	localStorage
courier	Intelligence	Medium	Growth+	localStorage
sku	Intelligence	Medium	Growth+	localStorage
campaigns	Intelligence	Medium	Pro	localStorage
address	Infrastructure	Low	All	localStorage
brand	Infrastructure	Low	All	localStorage + DB
plans	Infrastructure	Low	All	Config only
roles	Infrastructure	Low	All	Config only
stores	Infrastructure	Low	Pro	localStorage
onboarding	Infrastructure	Low	Pro	localStorage
privacy	Infrastructure	Low	All	lib only
integration-readiness	Infrastructure	Low	Pro	Static
sops	Infrastructure	Low	Pro	Static
learning	Content	Low	All	Static
marketing	Content	Low	Public	Static
demo	Testing	High	All	localStorage

---

4. DATABASE MAP

Deployed (Prisma) vs Intended (DATA_MODEL.md)

Table	Deployed?	Missing Fields in Deployed Schema
brands	⚠ Partial	forward_shipping_cost, return_shipping_cost, packaging_cost, estimated_cac, cod_fee, gross_margin_percent, risk_thresholds, courier_platforms, currency
users	⚠ Partial	name field missing
brand_members	✗ Missing	—
imports	✗ Missing	filename, source_type, row_count, success_count, error_count
orders	⚠ Partial	Missing: order_date, customer_name, full_address, landmark, pincode, city, state, sku, product_name, quantity, order_value, courier, campaign_name, ndr_reason, attempt_count, final_status, recommended_action, raw_data
customers	✗ Missing	—
risk_scores	✗ Missing	—
address_checks	✗ Missing	—

Table	Deployed?	Missing Fields in Deployed Schema
ndr_cases	⚠ Partial	Missing: awb, courier, ndr_reason_raw, ndr_reason_normalized, confidence, attempt_count, customer_response_status, recommended_action, action_status, final_outcome
messages	❌ Missing	—
customer_responses	❌ Missing	—
actions	⚠ Partial	Missing: assigned_to, notes, completed_at, confidence, estimated_leakage
savings_events	⚠ Partial	Missing: source_feature, formula_note, confidence, calculation_json
audit_logs	⚠ Partial	Missing: entity_type, metadata_json

Summary: 6/14 tables deployed, all partially. The DB is a skeleton of the intended schema.

---

5. API INVENTORY

Endpoint	Methods	Auth	Validation	Notes
`/api/v1/orders`	GET, POST (import)	Supabase JWT	orderImportSchema	No DELETE
`/api/v1/orders/[id]`	GET, PATCH	Supabase JWT	orderUpdateSchema	—
`/api/v1/ndr`	GET, POST	Supabase JWT	ndrCaseCreateSchema	—
`/api/v1/ndr/[id]`	GET, PATCH	Supabase JWT	ndrCaseUpdateSchema	—
`/api/v1/actions`	GET, POST	Supabase JWT	actionCreateSchema	Filter by orderId/ndrCaseId
`/api/v1/actions/[id]`	GET, PATCH	Supabase JWT	actionUpdateSchema	Status only
`/api/v1/savings`	GET, POST	Supabase JWT	savingsEventCreateSchema	Returns total + breakdown
`/api/v1/brand`	GET, PATCH	Supabase JWT	brandUpdateSchema	Name only
`/api/v1/audit`	GET, POST	Supabase JWT	auditLogCreateSchema	max 500 results
`/api/v1/users/me`	GET, POST	Supabase JWT	onboardSchema	POST creates Brand + User

Missing: DELETE on any resource, pagination params, search/filter params, webhook receivers

---

6. EVENT SYSTEM MAP

26 event types, stored in localStorage (not persistent):

```
brand.updated
store.created / store.updated
csv.imported
order.created / order.updated
risk.score.calculated
address.checked
ndr.detected
custom.rule.evaluated
action.created / action.completed / action.dismissed
message.queued / message.status.updated
customer.response.recorded
savings.event.created
policy.simulation.created
policy.recommendation.generated
weekly.report.generated
monthly.strategy.generated
export.created
data.deleted
```

Critical gap: No EventLog table in Prisma. All events are ephemeral. Restart browser = lose event history.

---

7. AUTOMATION MAP

Automation	Status	Trigger	Implementation
Risk scoring on import	✅ Live	CSV upload	`importPipeline` connector → `scoreOrder()`
NDR detection on import	✅ Live	CSV upload	`importPipeline` → `buildNdrCases()`
Address quality on import	✅ Live	CSV upload	`importPipeline` → `checkAddressQuality()`
Action queue build	✅ Live	Any data change	`orderToActions` connector
Policy recommendations	✅ Live	Analytics run	`policyRecommendation` connector
Prepaid opportunity detection	✅ Live	Order analysis	`orderToPrepaid` connector
Mock WhatsApp send	⚠️ Mock	Manual trigger	`actionToMessaging` → localStorage
Real WhatsApp	❌ Not built	—	Phase 2
Scheduled cron	❌ Not built	—	No infrastructure
Background jobs	❌ Not built	—	No queue system
Webhook receivers	❌ Not built	—	No handlers
Courier status polling	❌ Not built	—	Phase 4

---

8. AI / INTELLIGENCE MAP

System	Type	Location	Inputs	Outputs
RTO Risk Scoring	Rule-based	`lib/riskScoring.ts`	payment_mode, address, pincode history, customer history, NDR reason	score 0-100, bucket, reasons[]
Advanced Risk	Rule-based + custom rules	`features/risk/advancedRisk.service.ts`	Base score + margin, discount, campaign, custom rules	Enhanced score + confidence
Address	Penalty		full_address, landmark, pincode,	score 0-100,

System	Type	Location	Inputs	Outputs
Quality	deduction	`lib/addressQuality.ts`	phone	issues[]
NDR Normalization	Pattern matching	`lib/ndr.ts`	raw ndr_reason string	normalized reason, confidence 0.45-0.9
Intent Detection	Keyword matching	`lib/responses.ts`	Customer response text	intent enum, confidence
Prepaid Scoring	Multi-factor additive	`features/prepaid/service.ts`	risk bucket, margin, customer type, previous history	opportunity score, max safe incentive
Policy Simulation	Scenario modeling	`features/policy-simulator/service.ts`	policy type, orders, cost assumptions	net estimated benefit
Leakage Atlas	7-driver ranking	`features/leakage-atlas/service.ts`	All orders + NDR cases	top driver, estimated leakage, route

No ML. No external AI APIs. All deterministic.

Future ML roadmap (from MODEL_ROADMAP.md):

1. Calibrated RTO Prediction Model
2. Uplift Model for Prepaid Incentives
3. Expected Profit Optimization
4. Hazard Model for Reattempt Timing
5. Contextual Bandit for NDR Action Selection
6. Courier Assignment Optimizer
7. Pincode Cluster Model

---

9. DEPENDENCY GRAPH

```
graph TD
    subgraph "Everything depends on"
        DT[@/types/domain]
        LR[@/lib/riskScoring]
        LP[@/lib/profitRecovery]
        LI[@/lib/roi]
        EVT[@/shared/events]
        STG[@/shared/storage]
    end

    subgraph "High-dependency lib functions"
        LP --> |imported by| F1[8 features]
        LR --> |imported by| F2[5 features]
        LI --> |imported by| F3[4 features]
    end

    subgraph "Connector dependency order"
        C1[importPipeline] --> C2[orderToRisk]
        C2 --> C3[orderToNdr]
        C3 --> C4[orderToAddress]
        C4 --> C5[orderToActions]
        C5 --> C6[orderToCustomRules]
        C6 --> C7[orderToPrepaid]
        C7 --> C8[actionToMessaging]
    end
```

The single most depended-upon function: `estimatedLeakageForOrder()` from `lib/profitRecovery.ts` — used by leakage-atlas, actions, missions, policy-recommendations, policy-simulator, pincode, courier, sku, campaigns. Any change to this formula ripples across 9 features.

---

10. TECH DEBT REPORT

Priority 1 — Production Blockers

Debt	File	Fix
TypeScript errors suppressed	`next.config.ts:13`	Remove `ignoreBuildErrors`, fix all TS errors
ESLint disabled	`next.config.ts:17`	Enable, fix warnings
localStorage as primary state	Everywhere	Migrate to server storage (Supabase)
Missing 8 Prisma tables	`prisma/schema.prisma`	Add imports, customers, risk_scores, address_checks, messages, customer_responses, brand_members, stores
No RBAC in API routes	`src/app/api/v1/*`	Add role check after getAuthContext()
No soft deletes	All repositories	Add deletedAt to Brand, Order, NDRCase, Action

Priority 2 — Scale Blockers

Debt	Fix
O(N²) risk scoring in CSV import	Pre-aggregate pincode/courier rates before scoring loop
batchUpsert = N queries	Use raw SQL bulk UPSERT or Prisma createMany
No pagination on getByBrandId()	Add limit/offset to all repository list methods
Monolithic 57KB dashboard	Split into route-based pages: `/dashboard/risk`, `/dashboard/ndr`, etc.

Priority 3 — Maintainability

Debt	Fix
Hardcoded thresholds (₹999, ₹1499, ₹2499, ₹3999)	Move to brand config / feature flags
12h SLA hardcoded in ndrCases.ts	NDR playbook SLA already configurable — use it
Duplicate isNdrOrder() exports	Single source in lib, not re-exported from 3 features
No persistent event store	Add EventLog table to Prisma
NDR state machine has no transition validation	Add transition matrix with guard clauses

---

11. SCALABILITY REPORT

Metric	Current Limit	Target	Blocker
Orders per import	10,000 (Pro)	50,000+	O(N²) scoring, localStorage ceiling
Orders per brand per month	5,000 (Pro)	50,000+	No pagination, full-scan queries
Concurrent brands	Unknown	1,000+	No load testing, no rate limiting
Dashboard load time	All data upfront	< 1s	No lazy loading, no streaming
DB write throughput	1 query per order	Bulk	batchUpsert implementation

Metric	Current Limit	Target	Blocker
Build time	cpus:1	Faster	workerThreads disabled

The architecture can support the current pilot customers (300-5,000 orders/month) without issues. The first scaling crisis will happen around 10,000+ orders/brand or 100+ concurrent brands.

---

12. SECURITY REPORT

Control	Status	Priority
Authentication	✅ Supabase JWT	Done
Multi-tenant data isolation	✅ brandId scoping in all repos	Done
PII masking (phone)	✅ maskPhone() / phoneHash()	Done
HTTPS	✅ Vercel default	Done
Secrets management	✅ Env vars	Done
RBAC enforcement (API)	❌ Missing	🔴 P1
RBAC enforcement (Data layer)	❌ Missing	🔴 P1
PII in localStorage (unencrypted)	⚠️ Risk	🟠 P2
Rate limiting on imports	❌ Missing	🟠 P2
Soft deletes / data recovery	❌ Missing	🟠 P2
Persistent audit log (tamper-proof)	❌ Missing	🟡 P3
Input validation in lib functions	❌ Missing	🟡 P3
SQL injection protection	✅ Prisma parameterized	Done
XSS protection	✅ React default escaping	Done

---

13. MISSING SYSTEMS REPORT

System	Business Impact	Effort
Server-side state store — migrate from localStorage	Cannot launch B2B product without this	L (2 weeks)
8 missing Prisma tables	No server-side imports, messages, customer records	M (1 week)
Webhook receivers (WhatsApp, Shopify, Courier)	Required for Phase 2–4 automation	L (per integration)
Background job queue (BullMQ / Inngest)	CSV processing for 10k+ rows, async scoring	M (1 week)
Cron jobs — daily digest, SLA monitors	"NDR cases approaching SLA breach" alerts	S (2 days)
Email notifications	Ops team digest, alert on critical risk spike	S (2 days)
Pagination API	Required before first enterprise customer	S (2 days)
Search on orders/NDR	Ops teams search by order ID or phone	S (2 days)
DELETE endpoints	Data governance requirement	S (1 day)
Rate limiting	Prevent abuse and runaway CSV imports	S (1 day)

System	Business Impact	Effort
Observability (logging, metrics, tracing)	Invisible in production	M (1 week)
EventLog table	Compliance, debugging, replay	S (2 days)
CI/CD pipeline	No automated testing before deploy	S (1 day)
Health check endpoint	Monitoring and uptime tracking	S (hours)

---

14. FUTURE ARCHITECTURE RECOMMENDATIONS

Near-term (before first paid production customer)

```
1. Move workspace state → Supabase (Row-Level Security for multi-tenancy)
2. Complete Prisma schema (add 8 missing tables + missing fields)
3. Split monolithic dashboard → route-based pages
4. Add RBAC enforcement at API route level
5. Add soft deletes + EventLog table
6. Fix TypeScript + ESLint suppression
7. Add pagination to all repository list methods
8. Implement async CSV processing (stream parse → score → batch upsert)
```

Medium-term (Phase 2 — WhatsApp integration)

```
1. Add webhook receiver infrastructure (/api/webhooks/whatsapp)
2. Add background job queue (Inngest or BullMQ on Upstash)
3. Add message delivery status tracking
4. Add response capture automation
5. Build customer profile table (phone_hash-based repeat detection)
```

Long-term (Phase 5–6 — ML + Moat)

```
1. Export labeled outcome data for model training
2. Build feature store for risk signals
3. Add A/B testing infrastructure for model evaluation
4. Build cross-brand aggregated intelligence layer
5. Add courier × pincode performance index (requires N brands)
```

---

15. "IF I WERE CTO" PRIORITY LIST

Week 1 — Unblock Production

1. 🔴 Remove `ignoreBuildErrors` and fix all TypeScript errors
2. 🔴 Add RBAC check to all `/api/v1/` routes (2 lines per route)
3. 🔴 Add `deletedAt` soft delete to Brand, Order, NDRCase, Action
4. 🔴 Add a health check endpoint `/api/health`
5. 🔴 Add rate limiting middleware (Upstash Ratelimit or Vercel middleware)

Week 2 — DB Foundation 6. Write Prisma migrations for the 8 missing tables 7. Add missing fields to deployed tables (especially Orders — needs 20+ fields) 8. Add `(brand_id, order_id)`, `(brand_id, pincode)`, `(brand_id, final_status)` indexes 9. Implement paginated `getByBrandId(limit, offset)` across all repos

Week 3 — Split the Monolith 10. Extract dashboard views into separate route pages (`/dashboard/risk` , `/dashboard/ndr` , etc.) 11. Replace `useAppData` monolithic fetch with per-page lazy loading 12. Move CSV processing off main thread (Web Worker or server action + stream)

Week 4 — State Migration 13. Design Supabase RLS policies for multi-tenant workspace state 14. Build migration script: localStorage → Supabase for each entity 15. Run parallel writes (localStorage + DB) before cutting over 16. Add EventLog table + server-side event publishing

Month 2 — Observability + Automation Foundation 17. Add structured logging to all API routes (traceId, action, duration, brandId) 18. Add Inngest or similar for background jobs (CSV scoring, SLA monitors) 19. Add daily digest cron (critical NDR cases + action queue summary) 20. Ship CI/CD (GitHub Actions → Vercel preview → production)

COMPLETE MENTAL MODEL OF THE COMPANY

```
WEMBRO IS AN ECOMMERCE PROFIT RECOVERY OPERATING SYSTEM.

THE BUSINESS ENGINE:
  Indian D2C seller ships COD orders → 15-30% return to origin →
  ₹300-600 lost per failed delivery → seller has no visibility →
  Wembro imports their order data → scores every COD order for
  RTO risk → builds a daily action queue → rescues NDR cases
  before the courier marks them RTO → records every rupee saved
  → proves ROI → seller pays ₹2,999-14,999/month.

THE TECHNICAL SYSTEM:
  Next.js 15 frontend (one 57KB dashboard component) →
  Rule-based intelligence layer (34 feature modules, 13 connectors,
  26 events, 0 ML) → localStorage state (MVP) + partial Prisma DB
  (6/14 tables) → Supabase auth + Vercel hosting.

THE MOAT:
  TODAY: Workflow stickiness (ops habit).
  TOMORROW: Cross-brand pincode/courier intelligence
  that no single seller can build alone.

THE SINGLE BIGGEST RISK:
  The gap between the current localStorage-based MVP and
  a production-grade multi-tenant SaaS is 4-6 weeks of
  foundational engineering work. That work must happen
  before the first paying customer goes live.

THE SINGLE BIGGEST OPPORTUNITY:
  The product is conceptually complete and demo-ready.
  The core loop (CSV → Risk → Action → NDR Rescue → Savings)
  is fully implemented. The gap is infrastructure, not product.
  Once the DB schema is complete and state is server-side,
  this is a shippable B2B SaaS.
```

Generated by systematic codebase reverse-engineering. Verified against: README.md, 14 docs files, prisma/schema.prisma, src/app/api/v1/(10 routes), src/lib/* (26 files), src/features/* (34 modules), src/shared/* (connectors, events, storage), src/backend/* (repos, schemas), tests/* (21 test files).*